Docker

Szkolenie Docker + Kubernetes • część 1 • ALX

O mnie



przemeknowak.com
github.com/pnowy
linkedin.com/in/przemeknowak

Przemek Nowak

Cloud Architect · DevOps · konsultant · Warszawa

- **GCP / AWS: on-prem → cloud, FinOps, AI workloads**
- **Docker / Kubernetes, CI/CD: GitHub, GitLab, Jenkins, ArgoCD**
- **IaC: Terraform, Crossplane, Helm · observability: Prometheus, Grafana, Loki, OpenTelemetry**
- **Go, Python, TypeScript, Java**

Certyfikaty

- AWS Solutions Architect – Professional
- Google Cloud Professional Cloud Architect
- CKA – Certified Kubernetes Administrator
- CKAD – Certified Kubernetes Application Developer

Plan części Docker

Dzień 1	Sekcja
9:15–9:50	1. Wprowadzenie
9:50–10:15	2. Instalacja
10:15–11:00	3. Zarządzanie obrazami
11:15–13:00	4. Dockerfile + lab
13:45–14:30	5. Zarządzanie kontenerami
14:30–15:00	6. Woluminy
15:00–15:45	7. Sieci
16:00–16:45	8. Docker Compose + lab
16:45–17:00	9. Ćwiczenia – podsumowanie

Dzień 2	Sekcja
9:00–9:45	10. Multi-stage builds
9:45–10:15	11. Bezpieczeństwo
10:15–10:45	12. Optymalizacja
11:00–12:00	13. Compose – zaawansowane
12:00–12:30	14. Prywatny rejestr
12:30–12:45	15. Troubleshooting
12:45–13:00	16. AI (dodatek)
od 13:45	**Kubernetes**

★ **przy tytule slajdu – materiał dodatkowy**

Aplikacja przewodnia: notes-api

- **proste API notatek: Node.js 24 + TypeScript + Express 5**
- **przechowywanie danych (`STORAGE`):**
  - `postgres` **– PostgreSQL 18 (domyślnie)**
  - `json` **– plik** `data/notes.json`**, bez bazy**
- **wersja w Pythonie: FastAPI + MariaDB (`app-python/`)** ★
- **dziś: obraz → kontener → docker compose**
- **jutro: ten sam obraz w Kubernetesie**

Endpoint	Opis
`GET /`	wersja i nazwa hosta
`GET /healthz`	proces działa (liveness)
`GET /readyz`	jest baza (readiness)
`GET /notes`	lista notatek
`POST /notes`	`{"text": "..."}`

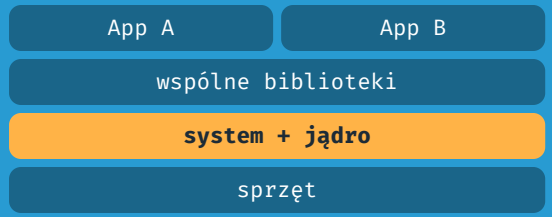
1. Wprowadzenie do kontenerów i Dockera

od serwera do kontenera · historia · architektura

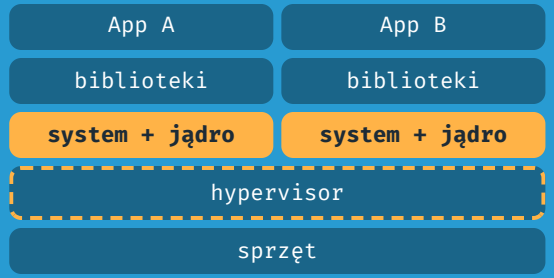
Infrastruktura – jak to się zmieniało

Kiedy	Zmiana	Co rozwiązała	Nowy problem
1990	mainframe → **PC / serwery x86**	tani sprzęt, każdy dział ma serwer	jedna aplikacja na serwer, 10–15% użycia
2000	serwer fizyczny → **wirtualizacja**	wiele systemów na jednym sprzęcie	każda VM to cały system do utrzymania
2010	serwerownia → **chmura**	VM w minutę, płacisz za użycie	„u mnie działa” – różne środowiska
2014	host → **kontenery**	ten sam obraz dev → test → prod	setki kontenerów na wielu maszynach
2015+	kontenery → **orkiestracja**	Kubernetes: skalowanie, self-healing	złożoność platformy

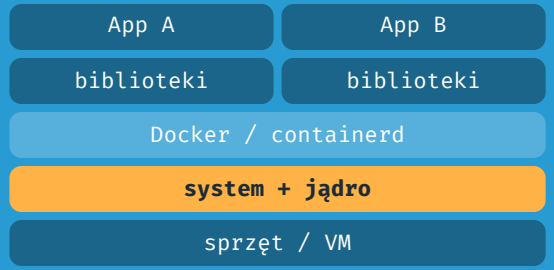
Trzy sposoby uruchomienia aplikacji



Serwer fizyczny



Wirtualizacja



Konteneryzacja

Wirtualizacja – hypervisor

Typ	Działa na	Przykłady
1 – bare metal	sprzęcie	VMware ESXi, Hyper-V, KVM, Xen
2 – hostowany	systemie hosta	VirtualBox, VMware Workstation/Fusion, Parallels, UTM

Rok	Kamień milowy
1967	IBM CP-67 – wirtualizacja na mainframe
1999	VMware Workstation – maszyny wirtualne na x86
2003	Xen – open source, później podstawa AWS EC2
2005–06	Intel VT-x / AMD-V – wsparcie wirtualizacji w CPU
2006	AWS EC2 – VM na żądanie, początek chmury publicznej
2007	KVM w jądrze Linuksa

Kontener a maszyna wirtualna

Maszyna wirtualna

- wirtualizuje sprzęt
- każda VM ma własne jądro i system
- GB pamięci, start w minutach
- silna izolacja – granica sprzętowa

Kontener

- wirtualizuje system operacyjny
- wspólne jądro hosta – to tylko proces
- MB pamięci, start w ułamku sekundy
- izolacja na poziomie jądra

Historia kontenerów – 40 lat jednego pomysłu

Rok	Technologia	Co dodała
1979	`chroot` (Unix V7)	proces widzi inny katalog jako `/`
2000	FreeBSD **jails**	osobne procesy, sieć i użytkownicy
2004	Solaris **Zones**	pełne „kontenery” systemowe
2002–2013	Linux **namespaces**	mnt (2002) … pid, net (2006–08) … user (2013)
2006–2008	**cgroups** (Google)	limity CPU / RAM dla grup procesów, jądro 2.6.24
2008	**LXC**	namespaces + cgroups razem, ale trudne w użyciu
2013	**Docker**	obraz + rejestr + proste CLI = kontener dla programisty
2015	**OCI** (Open Container Initiative)	standard obrazu i runtime’u – `runc`

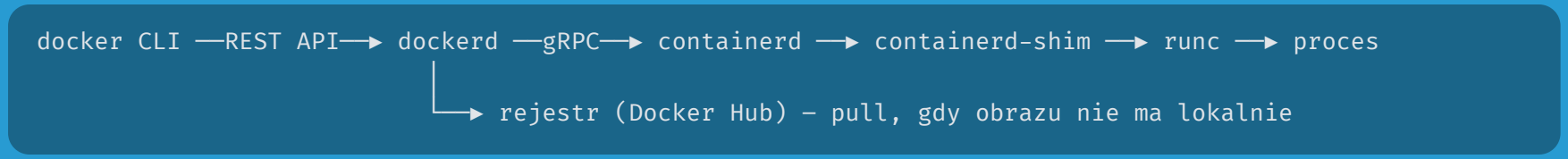
Docker – historia projektu

Rok	Wydarzenie
2010	dotCloud – PaaS, wewnętrznie kontenery LXC
2013	PyCon: 5-minutowe demo Solomona Hykesa, open source
2014	Docker 1.0, własny `libcontainer` zamiast LXC
2015	OCI, `runc` oddany społeczności
2017	`containerd` w CNCF, projekt Moby
2019	Docker Enterprise sprzedany (Mirantis) – fokus na deweloperach
2021	Docker Desktop płatny w firmach > 250 osób lub > 10 mln USD

Docker dziś

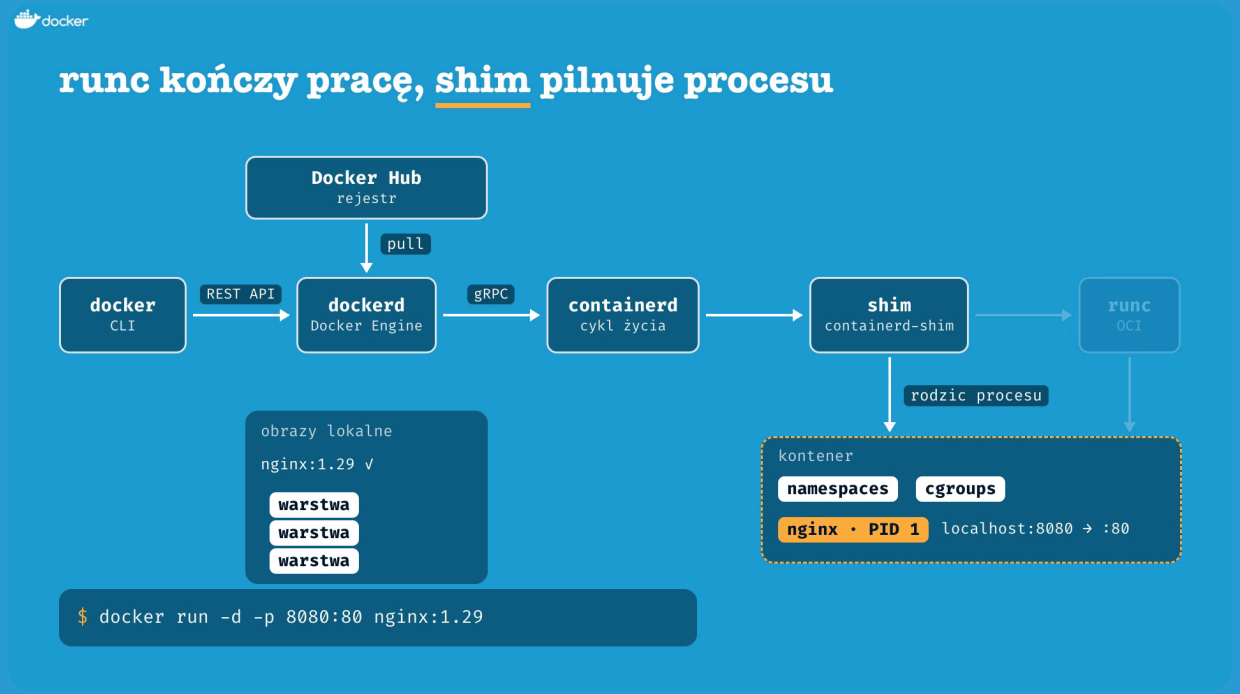
- **Docker Desktop, Docker Hub, Scout, Build Cloud**
- **obrazy Dockera = obrazy OCI → działają w Kubernetesie**
- **alternatywy: Podman, OrbStack, Colima, Rancher Desktop**

Architektura Dockera



Komponent	Rola
docker CLI	klient – wysyła polecenia do API (`/var/run/docker.sock`)
dockerd	API, obrazy, sieci, woluminy, budowanie (BuildKit)
containerd	cykl życia kontenerów – ten sam runtime używa Kubernetes
shim	rodzic procesu – trzyma stdio i kod wyjścia (1 na kontener)
runc	tworzy namespaces i cgroups, uruchamia proces (standard **OCI**)

Co się dzieje przy `docker run`



animacja: assets/animations/architektura.mp4

Kluczowe pojęcia

Obraz – szablon tylko do odczytu: system plików + konfiguracja

Kontener – uruchomiona instancja obrazu (proces + warstwa zapisu)

Rejestr – serwer z obrazami (Docker Hub, GHCR, Harbor)

Daemon – `dockerd`, zarządza obrazami, kontenerami, sieciami, woluminami

2. Instalacja Dockera

Ubuntu: apt lub get.docker.com • Docker Desktop • kontener to tylko proces

Gdzie działa Docker na Twoim komputerze

System	Co instalujesz	Gdzie są kontenery
Linux	Docker Engine (apt)	bezpośrednio na hoście – `ps` hosta widzi ich procesy
macOS	Docker Desktop	VM z Linuksem (Docker VMM / Apple Virtualization)
Windows	Docker Desktop	VM WSL 2 (albo Hyper-V)

Na macOS i Windows na hoście działa tylko CLI – `dockerd`, containerd i kontenery są w VM.
`docker context ls` pokazuje, z którym daemonem rozmawia CLI.

Ćwiczenie 2A – sprawdzenie środowiska

```
docker version            # sekcje Client i Server – CLI łączy się z daemonem
docker info               # kontenery, obrazy, architektura
docker compose version    # plugin "docker compose"
docker run --rm hello-world
```

`hello-world` : **CLI → daemon → brak obrazu → pull z Docker Hub → utworzenie kontenera → uruchomienie → wynik w terminalu**

Wygodny terminal na Linuksie ★

bash – autouzupełnianie wystarczy

```
sudo apt-get install -y bash-completion
docker con<TAB><TAB>   # config container context
```

zsh + oh-my-zsh – wtyczka `docker`

```
sudo apt-get install -y zsh git
sh -c "$(curl -fsSL https://install.ohmyz.sh)" \
  "" --unattended
# ~/.zshrc: plugins=(git docker)
```

starship – prompt z kontekstem Dockera

```
curl -sS https://starship.rs/install.sh \
  | sh -s -- -y
echo 'eval "$(starship init zsh)"' >> ~/.zshrc
```

```
# ~/.config/starship.toml
[docker_context]
only_with_files = false
```

```
~/app via 🐳 zdalny ❯
```

Kontener to tylko proces – jak to zobaczyć ★

```
docker run -d --name proces busybox:1.37 sleep 600

docker top proces        # PID według hosta (VM)
docker exec proces ps    # PID 1 według kontenera

# Linux: zwykłe ps na hoście
ps -ef | grep "sleep 600"

# macOS / Windows: pomocniczy kontener
# ze wspólną przestrzenią PID z VM
docker run --rm --pid=host busybox:1.37 \
  ps -o pid,ppid,comm | grep -E "PID|sleep|shim"
```

Ten sam proces, dwa numery

```
NSpid:  14581   1
        VM      kontener
```

- **rodzic:** `containerd-shim` **– jak w animacji**
- **jądro** `…-linuxkit` **/** `…-WSL2` **= VM Docker Desktop**
- `--memory 64m` **→ plik** `memory.max` **w cgroups**

Ćwiczenie 2B – kontener to tylko proces ★

1. `sleep 600` w kontenerze – jaki ma PID według hosta, a jaki według siebie?
2. Jaką nazwę hosta i jakie jądro widzi kontener?
3. Gdzie w kontenerze widać limit `--memory 64m`?
4. Ile trwa start kontenera?

3. Zarządzanie obrazami

search • pull • inspect • history • tag • push • warstwy

Czym jest obraz

Warstwy

stos archiwów `tar` z plikami – system plików aplikacji, tylko do odczytu

Konfiguracja

JSON z metadanymi: `CMD`, `ENTRYPOINT`, `ENV`, `USER`, `WORKDIR`, porty

Manifest

spis warstw i konfiguracji; jego skrót `sha256` to digest obrazu

- obraz : kontener = program : proces – z jednego obrazu dowolnie wiele kontenerów
- kontener = warstwy obrazu + cienka warstwa zapisu + izolowany proces
- obraz jest niezmienny – zmiana = nowy obraz (nowe warstwy, nowy digest)
- format OCI – ten sam obraz uruchomi Docker, Podman i Kubernetes (containerd)

Praca z obrazami

```
docker search --filter=stars=100 nginx
docker pull ubuntu:20.04                    # zawsze konkretny tag
docker images                               # = docker image ls
docker inspect ubuntu:20.04                 # konfiguracja (JSON)
docker history ubuntu:20.04                 # warstwy i instrukcje
docker tag ubuntu:20.04 moje-ubuntu:1.0     # nowa nazwa, to samo ID
docker push mojnazwa/moj-apache:1.0         # publikacja (po docker login)
docker rmi moje-ubuntu:1.0                  # usuwa tag, obraz zostaje pod ubuntu:20.04
docker image prune                          # nieużywane obrazy bez tagu
docker system df                            # zajętość: obrazy, kontenery, woluminy, cache
```

Nazwa, tag i digest

```
mojrejestr.example.com:5000 / zespol/notes-api : 1.0.0 @ sha256:cea1a360b945…
```

rejestr | repozytorium | tag | digest

- **bez rejestru →** `docker.io`**, bez tagu →** `latest`
- **tag jest ruchomy –** `latest`**,** `24` **wskazują różne wersje w czasie**
- **digest = skrót treści, zawsze ten sam obraz**
- **jeden tag → lista manifestów dla wielu architektur (amd64, arm64)**

```
docker buildx imagetools inspect ubuntu:20.04
```

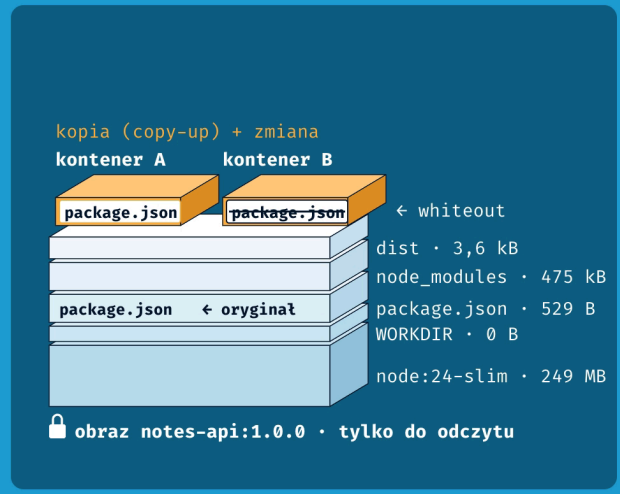
Warstwy i copy-on-write

Usunięcie pliku: znacznik whiteout w warstwie B

Dockerfile (etap runtime)

```
FROM node:24-slim
WORKDIR /app
COPY package.json ./
COPY node_modules ./node_modules
COPY dist ./dist
CMD ["node", "dist/server.js"]
# CMD = metadane, bez warstwy
```

```
$ docker exec B rm package.json
```



animacja: assets/animations/warstwy.mp4

Warstwy w praktyce ★

```
docker run -d --name w1 nginx:1.29
docker run -d --name w2 nginx:1.29
docker exec w1 sh -c 'echo zmiana > /usr/share/nginx/html/index.html'
docker exec w1 rm /usr/share/nginx/html/50x.html
docker diff w1              # C – zmieniony, A – dodany, D – usunięty
docker ps -s                # SIZE: warstwa zapisu, virtual: + warstwy obrazu
docker rm -f w1 w2          # warstwy zapisu znikają
```

Usunięcie pliku w późniejszej warstwie nie zmniejsza obrazu – plik nadal jest w warstwie, w której powstał.

Ćwiczenie I – pobieranie i inspekcja obrazu MariaDB

```
docker search mariadb
docker pull mariadb
docker images mariadb
docker inspect mariadb
docker history mariadb
```

Dodatkowe pytania

1. **Jakie porty, woluminy i polecenie startowe definiuje obraz?**
2. **Która instrukcja dodała najwięcej danych?**
3. **Jaki jest digest obrazu?**

`docker/09-cwiczenia/cwiczenie-1-obraz-mariadb/ · ok. 10 min`

docker

Przerwa

wracamy o 11:15

4. Dockerfile

instrukcje • build • ENTRYPOINT vs CMD • build checks

Najważniejsze instrukcje

Instrukcja	Opis
`FROM`	obraz bazowy, początek etapu
`RUN` · `COPY` · `ADD`	polecenie podczas budowania / kopiowanie plików – **tworzą warstwy**
`WORKDIR` · `ENV` · `ARG`	katalog roboczy / zmienne (runtime) / zmienne budowania
`USER`	użytkownik dla kolejnych instrukcji i procesu
`EXPOSE`	dokumentacja portu
`CMD` · `ENTRYPOINT`	polecenie startowe
`HEALTHCHECK`	sprawdzanie stanu → `healthy` / `unhealthy`
`VOLUME` · `STOPSIGNAL`	punkt montowania / sygnał zatrzymania

Przykład – Apache na Ubuntu 20.04

```
FROM ubuntu:20.04
LABEL maintainer="twoj.email@example.com"
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y apache2 curl && \
    apt-get clean && rm -rf /var/lib/apt/lists/*
COPY index.html /var/www/html/
WORKDIR /var/www/html
EXPOSE 80
HEALTHCHECK --interval=5s --timeout=3s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

```
docker build --check .                  # reguły BuildKit (lint)
docker build -t moj-apache:1.0 .        # "." = kontekst budowania
docker run -d -p 8080:80 moj-apache:1.0 && docker ps   # (healthy)
```

ENTRYPOINT + CMD = polecenie startowe ★

Dockerfile	`docker run obraz`	`docker run obraz ALX`
`CMD ["echo", "Domyślne"]`	`echo Domyślne`	`ALX`
`ENTRYPOINT ["echo", "Witaj,"]` + `CMD ["świecie"]`	`echo Witaj, świecie`	`echo Witaj, ALX`

- **CMD – domyślne polecenie; argumenty `docker run` zastępują je w całości**
- **ENTRYPOINT – stały program; CMD to jego domyślne argumenty**
- `docker run --entrypoint echo obraz Hello` **– nowy program, CMD z obrazu znika**

Brak ENTRYPOINT i dziedziczenie ★

- **bez ENTRYPOINT Docker uruchamia samo CMD – `docker run obraz sh` uruchomi powłokę zamiast aplikacji**
- **obraz dziedziczy ENTRYPOINT i CMD z `FROM`:**

Obraz	ENTRYPOINT	CMD
`ubuntu:24.04`	–	`["/bin/bash"]`
`node:24-slim`	`["docker-entrypoint.sh"]`	`["node"]`
`nginx:1.29`	`["/docker-entrypoint.sh"]`	`["nginx", "-g", "daemon off;"]`

- **ustawienie ENTRYPOINT kasuje odziedziczone CMD – podaj CMD ponownie**

```
docker image inspect -f '{{json .Config.Entrypoint}} {{json .Config.Cmd}}' node:24-slim
```

Forma exec [ ] vs forma shell ★

```
CMD ["node", "dist/server.js"]   # exec: node = PID 1
CMD node dist/server.js          # shell: ["/bin/sh", "-c", "node dist/server.js"]
```

Cecha	exec `["…"]`	shell (bez nawiasów)
PID 1	Twój program	`/bin/sh`, program jest dzieckiem
SIGTERM z `docker stop`	dociera do programu	**nie dociera** (sh go nie przekazuje)
`$VAR`, `&&`, potoki	nie działają	działają

Zasada: zawsze forma exec

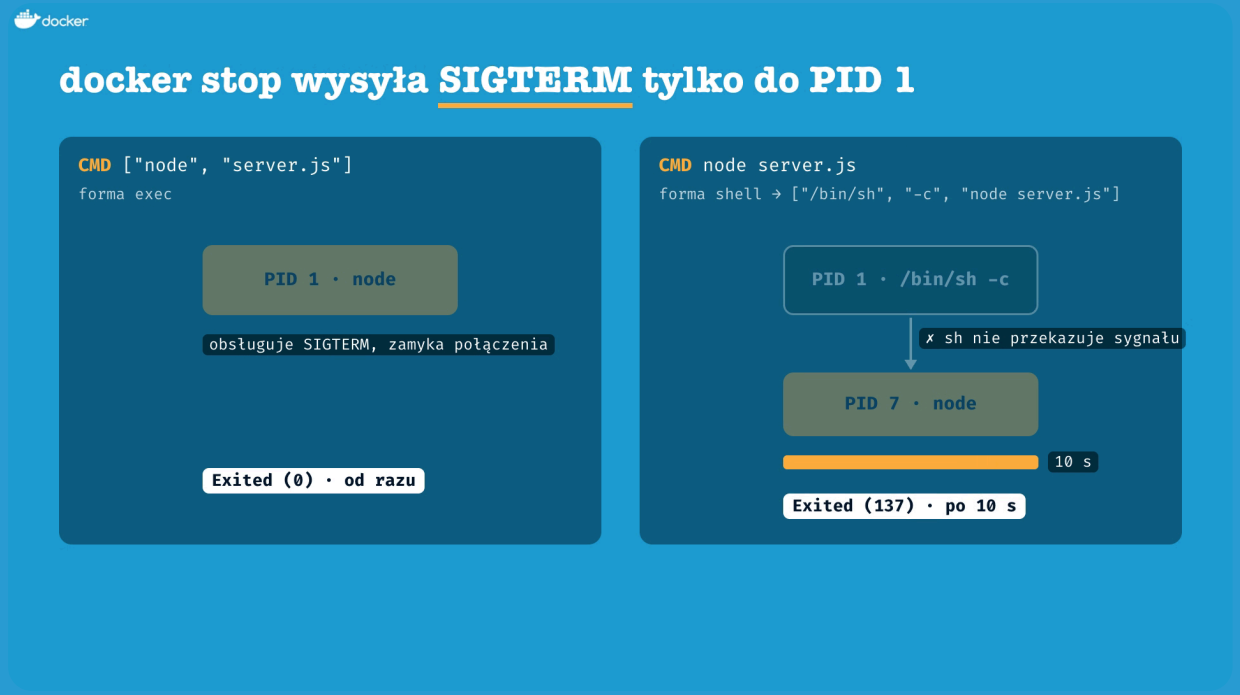
Skrypt startowy – `entrypoint.sh` ★

```
FROM notes-api:json
COPY --chmod=755 entrypoint.sh /usr/local/bin/
ENTRYPOINT ["entrypoint.sh"]
CMD ["node", "dist/server.js"]
```

```
#!/bin/sh
set -e
# przygotowanie: konfiguracja, dane startowe…
echo "uruchamiam $*"
exec "$@"
```

- **skrypt dostaje CMD jako argumenty:** `"$@"` = `node dist/server.js`
- `exec` **zastępuje powłokę aplikacją → aplikacja jest PID 1 i dostaje SIGTERM**
- **bez** `exec` **PID 1 zostaje** `sh` **→** `docker stop` **kończy się SIGKILL (137)**
- **tak działają oficjalne obrazy** `node`, `nginx`, `postgres`

ENTRYPOINT, CMD i sygnały – animacja ★



animacja: assets/animations/entrypoint-cmd.mp4

SIGTERM i `docker stop` – teoria ★

1. `docker stop` **wysyła do PID 1 sygnał SIGTERM**
2. **czeka na zakończenie (domyślnie 10 s,** `docker stop -t`**; Kubernetes: 30 s)**
3. **potem SIGKILL – proces ginie natychmiast**

Dockerfile	PID 1	Wynik
`CMD ["node", "dist/server.js"]`	`node dist/server.js`	od razu, kod 0
`CMD node dist/server.js`	`/bin/sh -c node …`	po limicie, kod 137
skrypt startowy z `exec "$@"`	`node dist/server.js`	od razu, kod 0
program bez obsługi SIGTERM	`node -e …`	po limicie, kod 137
to samo + `docker run --init`	`/sbin/docker-init -- …`	od razu, kod 143

PID 1 ignoruje sygnały bez własnej obsługi · 137 = SIGKILL · 143 = SIGTERM

Build checks i dobre praktyki

```
docker build --check .            # JSONArgsRecommended, SecretsUsedInArgOrEnv, UndefinedVar…
docker --debug build --check .    # z linkiem do opisu reguły
```

- **konkretne tagi obrazów bazowych, nie** `latest`
- `apt-get update && install … && rm -rf /var/lib/apt/lists/*` **w jednym** `RUN`
- `COPY` **zamiast** `ADD`**, forma exec w** `CMD` **/** `ENTRYPOINT`
- `USER` **– nie root**
- `.dockerignore`

Ćwiczenie II + lab – konteneryzacja notes-api

Ćwiczenie II (15 min) – Dockerfile z sekcji 4

1. `LABEL maintainer="twoj.nowy@email.com"`
2. `alx.html` + `COPY alx.html /var/www/html/`
3. `docker build -t moj-apache:2.0 .`
4. `docker run -d -p 8081:80 --name test-apache moj-apache:2.0`
5. `curl http://localhost:8081/alx.html`

Lab (30 min) – `app-node` albo `app-python` ★

1. **jak aplikacja się buduje i uruchamia?**
2. **napisz własny Dockerfile**
3. **porównaj z** `Dockerfile.simple`
4. `docker run -p 8000:3000` **(Python:** `8000:8000`**)** → `/`, `/readyz`
5. `--build-arg STORAGE=json` → `POST /notes` **bez bazy**

ćw. II: docker/09-cwiczenia/cwiczenie-2-dockerfile/start/ · lab: docker/04-dockerfile/README.md

Obiad

wracamy o 13:45

5. Zarządzanie kontenerami

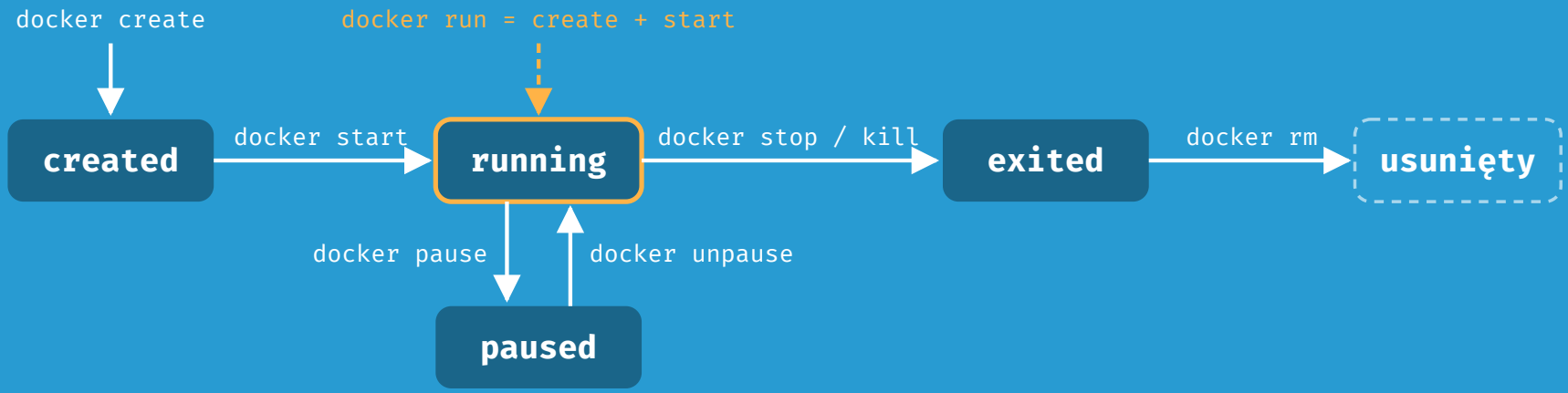
docker run • cykl życia • operacje • restart • limity • logi

docker run

```
docker run -d --name moj-serwer-www -p 8080:80 moj-apache:1.0   # w tle, port hosta 8080 → 80
docker run -it --rm ubuntu:20.04 /bin/bash                      # interaktywnie, usuń po wyjściu
docker run -d --memory=512m --cpus=1.0 --name api-server moj-apache:1.0
docker run --rm -e IMIE=Ala ubuntu:20.04 sh -c 'echo "Cześć $IMIE"'
```

Opcja	Znaczenie
`-d` / `-it`	w tle / interaktywnie z terminalem
`-p host:kontener`	publikowanie portu
`--name`, `--rm`	nazwa / usuń po zakończeniu
`-e`, `-v`, `--network`	zmienna, wolumin, sieć
`--restart`, `--memory`, `--cpus`	polityka restartu, limity

Cykl życia kontenera



- `docker run` = `pull` + `create` + `start`
- **kontener żyje tak długo jak jego proces główny (PID 1)**
- `docker stop` → **SIGTERM, po 10 s SIGKILL (Docker Desktop: 3 s)**
- **zatrzymany kontener zachowuje swoją warstwę zapisu do** `docker rm`

Podstawowe operacje

```
docker ps -a                                          # wszystkie kontenery
docker stop moj-serwer-www && docker start moj-serwer-www
docker logs --tail=50 moj-serwer-www                  # logi (stdout/stderr), -f na żywo
docker exec -it moj-serwer-www /bin/bash              # polecenie w działającym kontenerze
docker cp nowy_plik.html moj-serwer-www:/var/www/html/   # host → kontener
docker cp moj-serwer-www:/var/log/apache2/access.log .   # kontener → host
docker inspect -f '{{.State.Status}}' moj-serwer-www     # pojedyncze pole z inspect
docker stats --no-stream                              # zużycie CPU/RAM (bez odświeżania)
docker top moj-serwer-www                             # procesy w kontenerze
docker diff moj-serwer-www                            # zmiany w systemie plików kontenera
docker rm -f moj-serwer-www                           # zatrzymaj i usuń
```

Polityki restartu i limity zasobów ★

Polityka	Restart, gdy…
`no`	nigdy (domyślnie)
`on-failure[:N]`	kod ≠ 0, maks. N razy
`always`	zawsze, też po restarcie daemona
`unless-stopped`	jak always, ale nie po `docker stop`

```
docker run -d --memory=256m --cpus=0.5 nginx:1.29

# przekroczenie limitu pamięci
docker run --memory=64m busybox:1.37 \
  sh -c 'x=a; while true; do x=$x$x; done'
# OOMKilled=true, kod wyjścia 137
```

Logi i sterowniki logowania ★

Sterownik	`docker logs`	Uwagi
`json-file`	tak	domyślny w Engine, **bez rotacji**
`local`	tak	rotacja 5 × 20 MB, domyślny w Desktop
`journald`, `syslog`, `fluentd`, `awslogs`	tak*	*dual logging
`none`	nie	logi wyłączone

```
docker logs --tail 50 --since 10m --timestamps app
docker info --format '{{.LoggingDriver}}'
docker run -d --log-opt max-size=10m \
  --log-opt max-file=3 nginx:1.29
```

```
{
  "log-driver": "local",
  "log-opts": {
    "max-size": "10m", "max-file": "3"
  }
}
```

- **aplikacja pisze na stdout/stderr, nie do plików w kontenerze**
- `daemon.json` **dotyczy tylko nowych kontenerów**
- **Docker Desktop dokłada własne domyślne → zawsze** `docker info`

Ćwiczenie III – uruchamianie kontenerów MariaDB

```
docker run -d --name moja-baza -e MYSQL_ROOT_PASSWORD=tajnehaslo mariadb
docker inspect moja-baza | grep IPAddress
docker run -d --name moja-baza2 -p 3306:3306 -e MYSQL_ROOT_PASSWORD=tajnehaslo mariadb
docker exec -it moja-baza2 bash     # ps auxf · mariadb -u root -ptajnehaslo -e "SHOW DATABASES;"
docker diff moja-baza2
```

5A · dodatkowo – własna strona w nginx: `docker cp`**, logi,** `exec`**,** `diff` **– co przetrwa** `stop`**, a co** `rm`**?**

5B · dodatkowo – `--restart on-failure:3` **i OOM przy** `--memory 64m`

`docker/09-cwiczenia/cwiczenie-3-kontenery-mariadb/ · 15 min · 5A/5B: docker/05-kontenery`

6. Woluminy

trwałe dane poza warstwą zapisu kontenera

Woluminy – animacja



animacja: assets/animations/woluminy.mp4

Typy montowania

Typ	Gdzie są dane	Zastosowanie
named volume	zarządza Docker (`/var/lib/docker/volumes`)	bazy danych, dane aplikacji
bind mount	dowolny katalog hosta	development, konfiguracja
tmpfs	RAM, znika po zatrzymaniu	dane tymczasowe

```
docker volume create moje-dane          # ls · inspect moje-dane · rm moje-dane · prune
docker run -d --name moja-baza -e MYSQL_ROOT_PASSWORD=sekret -v moje-dane-db:/var/lib/mysql mariadb
docker run -d --name dev-server -v $(pwd)/src:/var/www/html -p 8080:80 moj-apache:1.0
docker run -d --name moja-baza --mount source=moje-dane-db,target=/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=sekret mariadb                # czytelniejsza składnia
docker run --rm --tmpfs /tmp/cache:size=64m ubuntu:20.04 df -h /tmp/cache
```

Woluminy – co warto wiedzieć

- **pusty wolumin dostaje zawartość katalogu z obrazu; bind mount ją przykrywa**
- **obrazy baz mają `VOLUME` – bez `-v` powstaje wolumin anonimowy → zawsze nadawaj nazwę**
- **PostgreSQL 18+: montuj `/var/lib/postgresql`**
- **kopia zapasowa przez kontener pomocniczy:**

```
docker run --rm -v moje-dane-db:/dane:ro -v "$(pwd):/backup" ubuntu:20.04 \
  tar czf /backup/moje-dane-db.tgz -C /dane .
```

Woluminy zdalne – NFS ★

```
docker volume create --driver local \
  --opt type=nfs --opt o=addr=nfs.firma.local,rw,nfsvers=4 --opt device=:/eksport/app dane-nfs
docker run --rm -v dane-nfs:/dane alpine mount | grep /dane     # type nfs4
```

```
volumes:
  dane-nfs:
    driver: local
    driver_opts: { type: nfs, o: "addr=nfs.firma.local,rw,nfsvers=4", device: ":/eksport/app" }
```

- **montuje daemon na hoście przy starcie kontenera – host potrzebuje klienta NFS**
- **SMB: `type=cifs`; inne systemy – wtyczki (`docker plugin install`)**
- **bazy na NFS – ryzykowne; w Kubernetesie: PersistentVolume + CSI**

Ćwiczenie IV – woluminy i trwałość danych

1. `docker volume create www-data`
2. `docker run -d --name moj-apache-wolumin -v www-data:/var/www/html/alx ubuntu_apache`
3. **Plik w woluminie:**

   Linux: `sudo touch /var/lib/docker/volumes/www-data/_data/alx.html`

   Docker Desktop: `docker run --rm -v www-data:/data busybox touch /data/alx.html`
4. `docker exec -it moj-apache-wolumin bash` → `ls /var/www/html/alx/`, `ps aux | grep apache2`, `kill <PID>`
5. `docker cp kopiowanie.html moj-apache-wolumin:/var/www/html/`
6. `docker rm -f …` → `nowy-apache` **z tym samym woluminem → co zostało?**

`docker/09-cwiczenia/cwiczenie-4-woluminy/ · ok. 15 min`

7. Sieci w Dockerze

sterowniki · DNS po nazwach · publikowanie portów

Sterowniki sieciowe

Sterownik	Zastosowanie
bridge	domyślny – prywatna sieć na hoście; w sieci użytkownika działa DNS po nazwach
host	kontener używa sieci hosta (w pełni tylko na Linuksie)
none	brak sieci, tylko `lo`
overlay	sieć między hostami – Docker Swarm (Kubernetes ma własny model: CNI)
macvlan	kontener z własnym adresem MAC w sieci fizycznej

```
docker network ls                                # bridge, host, none
docker network create moja-siec
docker network inspect moja-siec
docker network connect moja-siec <kontener>      # podłączenie działającego kontenera
docker network disconnect moja-siec <kontener>
docker network rm moja-siec
```

DNS – komunikacja po nazwach

```
docker network create moja-siec
docker run -d --name backend  --network moja-siec httpd:2.4
docker run -d --name frontend --network moja-siec -p 8080:80 nginx:1.29
docker exec frontend curl -s http://backend/      # nazwa kontenera = nazwa DNS
docker exec frontend cat /etc/resolv.conf          # nameserver 127.0.0.11
```

Sieć użytkownika – wbudowany DNS, aliasy

`--network-alias`

Domyślny bridge – brak DNS po nazwach, tylko IP

Publikowanie portów

```
docker run -d -p 8080:80 nginx:1.29            # wszystkie interfejsy hosta
docker run -d -p 127.0.0.1:8081:80 nginx:1.29  # tylko localhost
docker run -d -P nginx:1.29                    # losowe porty dla EXPOSE
docker port <kontener>
```

- `-p` = reguła NAT host → kontener
- `EXPOSE` w Dockerfile niczego nie publikuje – to dokumentacja
- kontenery w jednej sieci łączą się na porcie kontenera (`backend:80`)

Ćwiczenie V – sieć wielokontenerowa

1. `docker network create moja-app-siec`

2. **Baza:**

```
docker run -d --name baza --network moja-app-siec \
  -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=testdb mariadb
```

3. **Klient w tej samej sieci:**

```
docker run -it --rm --network moja-app-siec ubuntu bash
apt-get update && apt-get install -y mysql-client
mysql -h baza -u root -psecret testdb        # SELECT VERSION();
```

`docker/09-cwiczenia/cwiczenie-5-siec/ · ok. 15 min`

docker

Przerwa

wracamy o 16:00

8. Docker Compose

cała aplikacja jednym poleceniem

Instalacja i YAML w minutę

```yaml
services:                  # mapa: wcięcia spacjami
  api:
    ports:
      - "8080:3000"        # porty w cudzysłowie
    command: ["node", "dist/server.js"]
    environment:
      DEBUG: "false"       # zmienne to tekst
x-logi: &logi              # x-… pomijane, &kotwica
  driver: json-file
# … logging: *logi         # wstawienie kotwicy
```

- `docker compose` **(v2, wtyczka CLI) – w Docker Desktop wbudowany, na Ubuntu** `docker-compose-plugin`
- `docker-compose` **(v1, Python) – wycofany**
- `docker compose config` **wychwyci błędy składni**

compose.yaml – notes-api (Python) + MariaDB

```yaml
services:
  api:
    build: .                      # app-python/Dockerfile
    ports: ["8000:8000"]
    environment:                  # DB_HOST = nazwa serwisu bazy
      DB_HOST: db
      DB_NAME: notes
      DB_USER: notes
      DB_PASSWORD: haslo-szkoleniowe
    depends_on: [db]
    networks: [app-network]
  db:
    image: mariadb:11.8
    environment:
      MARIADB_ROOT_PASSWORD: sekret
      MARIADB_DATABASE: notes
      MARIADB_USER: notes
      MARIADB_PASSWORD: haslo-szkoleniowe
    volumes: ["db-data:/var/lib/mysql"]
    networks: [app-network]
volumes: { db-data: }
networks: { app-network: { driver: bridge } }
```

- **serwis = kontener (lub kilka replik) z tej samej konfiguracji**
- **nazwa serwisu = nazwa DNS – `api` łączy się z `db`**
- **`depends_on` czeka tylko na uruchomienie kontenera, nie na gotowość bazy**
- **bez `version: '3.8'` – w Compose v2 przestarzałe**
- **`compose.yaml` (stara nazwa `docker-compose.yml` nadal działa)**

Polecenia Compose

```
docker compose up -d                        # zbuduj (przy pierwszym uruchomieniu) i uruchom w tle
docker compose ps                           # stan serwisów
docker compose logs -f web                  # logi jednego serwisu
docker compose exec db mariadb -uroot -psekret -e "SHOW DATABASES;"
docker compose build                        # przebuduj obrazy
docker compose restart                      # restart serwisów
docker compose config                       # walidacja, wynik po scaleniu
docker compose down                         # dane zostają w woluminie
docker compose down -v                      # … razem z woluminami
```

projekt

nazwa katalogu albo `name:`

sieć

`<projekt>_default`

woluminy

`<projekt>_db-data`

Lab – aplikacja z bazą danych

```
cd app-python                          # compose.yaml: api (build .) + db (mariadb:11.8)
docker compose up -d --build
docker compose logs api
curl -X POST localhost:8000/notes -H 'Content-Type: application/json' -d '{"text":"Pierwsza"}'
curl localhost:8000/notes
docker compose exec db mariadb -unotes -phaslo-szkoleniowe notes -e 'SELECT * FROM notes;'
docker compose down && docker compose up -d
docker compose stop db && curl localhost:8000/readyz      # 503
docker compose start db
docker compose down -v
```

app-python/compose.yaml · docker/08-compose/compose-aplikacja.sh · ok. 30 min

9. Ćwiczenia praktyczne

ćwiczenia I–V · podsumowanie dnia 1

Ćwiczenia I–V – podsumowanie

Ćwiczenie	Temat	Sekcja
I	Pobieranie i inspekcja obrazu MariaDB	3. Obrazy
II	Modyfikacja Dockerfile	4. Dockerfile
III	Uruchamianie kontenerów MariaDB	5. Kontenery
IV	Woluminy i trwałość danych	6. Woluminy
V	Sieć wielokontenerowa	7. Sieci

Rozwiązania: `docker/09-cwiczenia/` • **kto nie skończył któregoś ćwiczenia – dokończ teraz**

10. Multi-stage builds

dzień 2 · małe obrazy produkcyjne

Multi-stage – przykład z FastAPI

```dockerfile
# ── Etap 1: Builder ──
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

# ── Etap 2: Runtime ──
FROM python:3.12-slim
WORKDIR /app
# tylko zależności, bez pip
COPY --from=builder /install /usr/local
COPY src/ .
# nie root!
USER nobody
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `AS builder` – **nazwa etapu**
- `COPY --from=builder` – **do obrazu trafia tylko wynik**
- **narzędzia budowania zostają w etapie** `builder`
- **domyślnie budowany jest ostatni etap**
- **komentarz zawsze w osobnej linii**

Multi-stage – notes-api (Node.js)

```dockerfile
FROM node:24-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev

FROM node:24-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci
COPY tsconfig.json ./
COPY src/ src/
RUN npm run build

FROM node:24-slim AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
USER node
CMD ["node", "dist/server.js"]
```

`RUN --mount` ≠ `docker run -v`

- **tylko na czas jednej instrukcji** `RUN`
- **nie trafia do warstwy obrazu**
- `cache` **– katalog npm/pip zostaje między buildami**
- `bind` **– pliki z kontekstu bez** `COPY`**, tylko odczyt**

`-v` **przy** `docker run` **– wolumin działającego kontenera (sekcja 6)**

Multi-stage – animacja



animacja: assets/animations/multi-stage.mp4

Efekt multi-stage

Dockerfile.simple

1,18 GB

node:24 · tsc · devDependencies · root

Dockerfile (multi-stage)

250 MB

node:24-slim · tylko dist/ · user node

```
docker build --target build -t notes-api:build .   # wybrany etap
```

Multi-stage – notes-api (Python) ★

```
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY src/ .
RUN useradd --uid 1000 app
USER app
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

Obraz	Rozmiar
`Dockerfile.simple`	1,15 GB
`Dockerfile` multi-stage	169 MB

- `--prefix=/install` → `COPY` **do** `/usr/local`, **gdzie obraz trzyma Pythona – bez zmiany** `PATH`
- **obraz** `python` **nie ma gotowego użytkownika –** `useradd`

`app-python/` · `docker/10-multi-stage/multi-stage-python.sh`

Wybór obrazu bazowego ★

Obraz	Rozmiar	Zawartość
`node:24`	1,14 GB	pełny Debian: gcc, git, curl – do budowania
`node:24-slim`	249 MB	minimalny Debian + Node.js – domyślny runtime
`node:24-alpine`	167 MB	musl zamiast glibc – uwaga na moduły natywne
`gcr.io/distroless/nodejs24-debian13`	162 MB	bez powłoki i menedżera pakietów
`scratch`	0 B	pusty – statyczne binarki (Go, Rust)

Sekrety podczas budowania ★

```
# ✗ ARG zostaje w "docker history"
ARG API_TOKEN

# ✓ sekret tylko na czas jednej instrukcji RUN
RUN --mount=type=secret,id=api_token,env=API_TOKEN ./pobierz-zaleznosci.sh
```

```
docker build --secret id=api_token,env=API_TOKEN -t moj-obraz .
```

- **BuildKit ostrzega:** `SecretsUsedInArgOrEnv`
- `RUN --mount=type=cache` **– cache npm/pip/apt między buildami, poza obrazem**

Obrazy wieloplatformowe ★

- **Mac z Apple Silicon →** `linux/arm64`**, serwery zwykle** `linux/amd64`
- **obraz z niewłaściwej architektury →** `exec format error`

```
docker build --platform linux/amd64 -t notes-api:amd64 .          # emulacja QEMU
docker buildx build --platform linux/amd64,linux/arm64 \
  -t <rejestr>/notes-api:1.0.0 --push .
```

Jeden tag dla wielu architektur lokalnie: containerd image store w Docker Desktop albo `docker buildx create --use`

Python i Java – te same zasady ★

```dockerfile
FROM python:3.14-slim AS build
RUN python -m venv /venv
ENV PATH=/venv/bin:$PATH
COPY requirements.txt ./
RUN pip install -r requirements.txt

FROM python:3.14-slim
ENV PATH=/venv/bin:$PATH PYTHONUNBUFFERED=1
COPY --from=build /venv /venv
COPY app.py ./
USER app
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

165 MB · gunicorn zamiast serwera dev

```dockerfile
FROM maven:3.9-eclipse-temurin-25 AS build
RUN mvn -q package -DskipTests
RUN java -Djarmode=tools -jar target/app.jar \
    extract --layers --destination /extracted

FROM eclipse-temurin:25-jre
COPY --from=build /extracted/dependencies/ ./
COPY --from=build /extracted/application/ ./
USER app
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"
CMD ["java", "-jar", "app.jar"]
```

379 MB (builder 520 MB) · kod w warstwie 4 kB

Do samodzielnego sprawdzenia – multi-stage

1. **Zbuduj i uruchom przykład FastAPI (`fastapi/build.sh`). Jako jaki użytkownik działa proces?**
2. **Porównaj rozmiar `notes-api:simple` z sekcji 4 z wersją multi-stage (`multi-stage.sh`).**
3. ★ **To samo dla wersji w Pythonie (`multi-stage-python.sh`).**

```
docker/10-multi-stage/ · ok. 15 min
```

11. Bezpieczeństwo kontenerów

non-root • hardening • skanowanie

Nie uruchamiaj jako root

```
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser          # albo w obrazach node: USER node
```

```
docker run -d --name hardened \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --pids-limit 100 --memory 256m \
  -p 8000:3000 notes-api:1.0.0
```

Nigdy bez potrzeby: `--privileged`**, montowanie** `/var/run/docker.sock`

Skanowanie podatności i SBOM

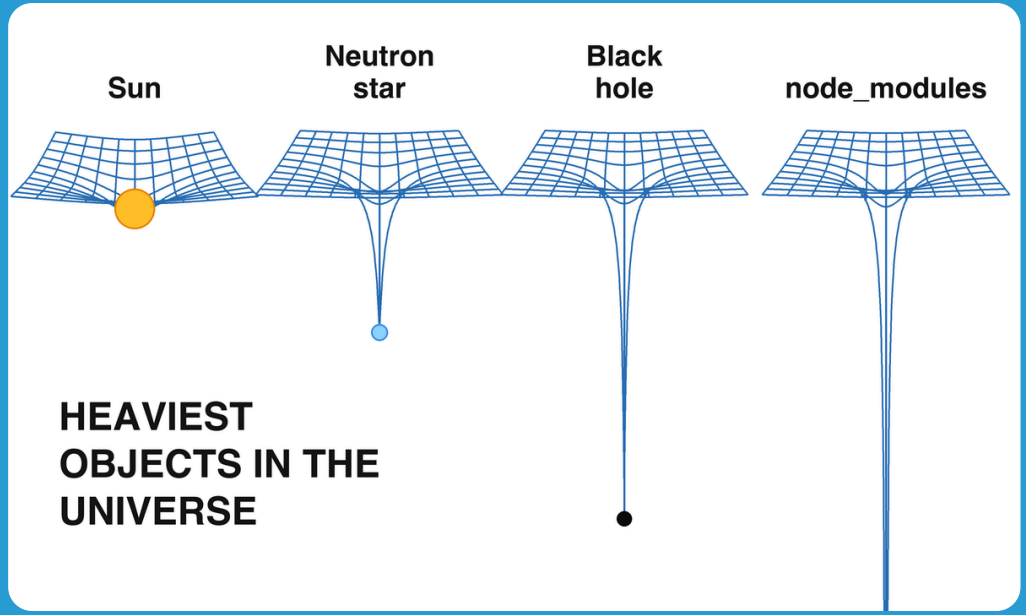
```
docker scout quickview notes-api:1.0.0
docker scout cves --only-severity critical,high notes-api:1.0.0
docker scout recommendations notes-api:1.0.0
trivy image notes-api:1.0.0
docker scout sbom --format list notes-api:1.0.0
```

- **minimalne obrazy bazowe, przypięte tagi lub digesty, regularna przebudowa**
- **sekrety: nie w `ENV` / `ARG` – `--mount=type=secret`, `secrets:` w Compose, Secret w K8s**

12. Optymalizacja rozmiaru obrazów

kolejność instrukcji · cache buildera · co zajmuje miejsce

Obraz bez .dockerignore

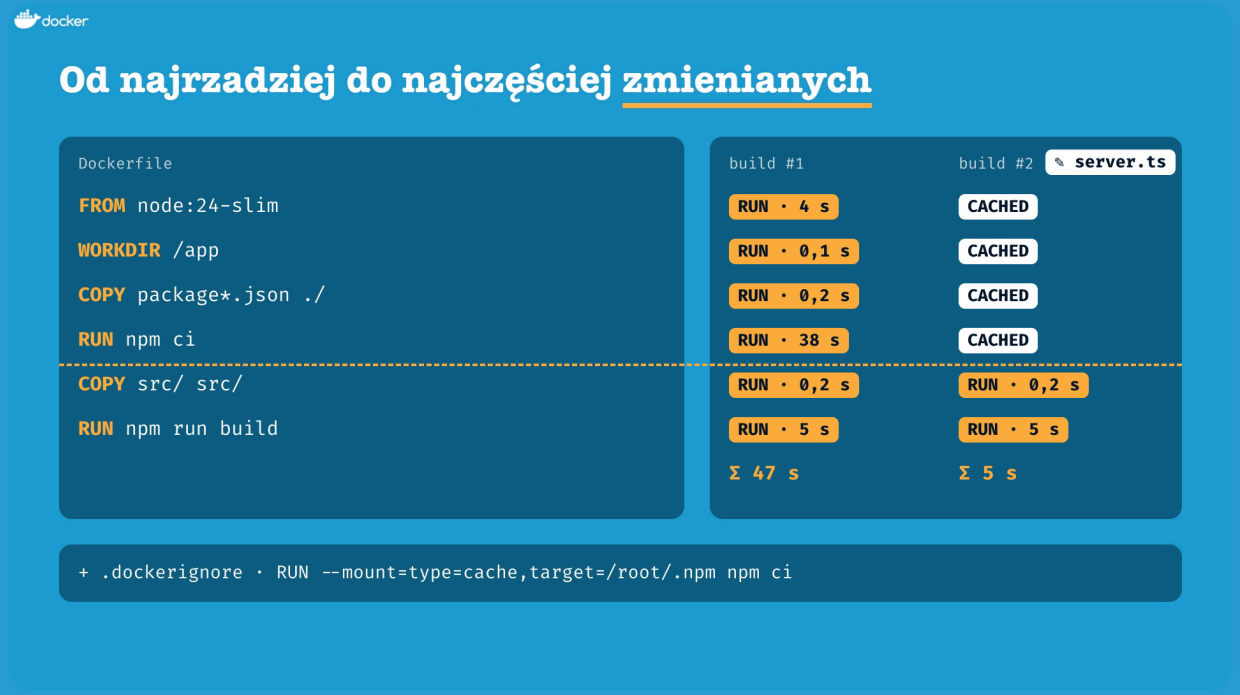


`COPY . .` **razem z** `node_modules`**,** `.git` **i logami.**

Cache buildera i kolejność instrukcji

- wynik każdej instrukcji jest w cache
- zmiana warstwy unieważnia wszystkie kolejne
- rzadko zmieniane – na górę, często – na dół
- `.dockerignore`: `node_modules`, `.git`, `.env`, `dist`

Cache w działaniu



animacja: assets/animations/cache.mp4

Co zajmuje miejsce w obrazie

```
docker history notes-api:cached          # rozmiar każdej warstwy
dive notes-api:cached                    # interaktywny eksplorator warstw (brew install dive)
docker system df                         # obrazy, kontenery, woluminy, cache buildera
docker buildx du | tail -3               # rozmiar cache BuildKit
docker build --progress=plain …          # pełny log budowania – widać CACHED
```

- **mniejszy obraz bazowy i multi-stage – sekcja 10**
- **sprzątanie w tym samym** `RUN` **:** `apt-get … && rm -rf /var/lib/apt/lists/*`
- **usunięcie pliku w kolejnej warstwie nie zmniejsza obrazu**
- `.dockerignore` **– mniejszy kontekst, mniej przypadkowych zmian w cache**

Host · obraz · kontener – kto za co odpowiada ★

Co	Gdzie
kod, zależności, runtime (Node.js, JRE)	**obraz** – niezmienny, wersjonowany
konfiguracja środowiska (adres bazy, poziom logów)	**kontener** – zmienne, montowane pliki
sekrety	**kontener** – secrets jako pliki
dane	**wolumin**
logi	stdout/stderr → sterownik logowania
jądro, Docker, poprawki, firewall, backup, monitoring	**host**
restart, repliki, aktualizacje	restart policy · Compose · Kubernetes

obraz buduj raz, konfiguruj przy uruchomieniu · jeden kontener = jeden proces główny

Do samodzielnego sprawdzenia – cache

W katalogu `app-node` – `Dockerfile.simple` i `Dockerfile.cached` mają te same instrukcje w innej kolejności:

1. **zbuduj oba obrazy (`notes-api:simple`, `notes-api:cached`)**
2. `echo "// zmiana" >> src/server.ts` **i zbuduj oba ponownie z** `--progress=plain`
3. **przy którym pliku** `npm ci` **wykonało się od nowa, a przy którym jest** `CACHED`**?**
4. `git checkout src/server.ts`

```
docker/12-optymalizacja/optymalizacja.sh · ok. 10 min
```

docker

Przerwa

wracamy o 11:00

13. Docker Compose – zaawansowane

.env · nadpisywanie · healthcheck · sekrety · produkcja · load balancing

`.env`, healthcheck i nadpisywanie plików

```yaml
# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5
  app:
    build: ../10-multi-stage/fastapi
    image: myapp:${APP_VERSION}
    environment:
      APP_VERSION: ${APP_VERSION}
    ports: ["8000:8000"]
    depends_on:
      db:
        condition: service_healthy
```

```yaml
# docker-compose.prod.yml
services:
  db:
    restart: always
  app:
    restart: always
    ports: !override
      - "80:8000"
```

```
docker compose config        # wartości z .env
docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up -d
```

- **kolejne pliki `-f` nadpisują poprzednie**
- **`!override` zastępuje listę zamiast ją łączyć**

Do samodzielnego sprawdzenia – Compose

1. `docker compose config` – **znajdź wartości podstawione z** `.env`
2. `docker compose up -d --build` **i** `docker compose ps` – **kiedy startuje** `app`**?**
3. `curl http://localhost:8000/` – **skąd aplikacja zna wersję?**
4. `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d` – **jakie porty?**
5. `docker compose -f docker-compose.yml -f docker-compose.prod.yml down`

```
docker/13-compose-zaawansowane/ · ok. 15 min
```

Pełna wersja notes-api – sekrety i profile ★

```yaml
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes: [db-data:/var/lib/postgresql]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
  api:
    build: { context: ., target: runtime }
    environment: { PGHOST: db, PGPASSWORD_FILE: /run/secrets/db_password }
    secrets: [db_password]
    ports: ["${APP_PORT}:3000"]
    depends_on: { db: { condition: service_healthy } }
volumes: { db-data: {} }
secrets: { db_password: { file: ./secrets/db_password.txt } }
```

Izolacja – kilka sieci w projekcie ★

```yaml
services:
  proxy: { networks: [frontend] }
  api:   { networks: [frontend, backend] }
  db:    { networks: [backend] }       # proxy nie dosięgnie bazy
networks:
  frontend:
  backend:
    internal: true                     # bez wyjścia do internetu
```

- **każdy projekt ma własną sieć** `<projekt>_default` **i woluminy** `<projekt>_…`
- **dwa projekty mogą mieć serwis** `db` **bez konfliktu**

Profile, nadpisywanie i compose watch ★

```
docker compose --profile narzedzia up -d                    # adminer tylko na żądanie
docker compose -f compose.yaml -f compose.prod.yaml up -d   # kolejne pliki nadpisują
docker compose -f compose.yaml -f compose.dev.yaml watch    # hot reload
```

```
# compose.dev.yaml
services:
  api:
    build: { target: dev }
    develop:
      watch:
        - { action: sync+restart, path: ./src, target: /app/src }
        - { action: rebuild, path: package.json }
```

- **pole `version:` jest przestarzałe – nie używamy**

Compose na produkcji ★

```yaml
# compose.prod.yaml (nadpisanie)
services:
  api:
    build: !reset null
    image: ${REGISTRY}/notes-api:${APP_VERSION}
    logging: *logi
    read_only: true
    deploy:
      resources:
        limits: { cpus: "0.5", memory: 256M }
    ports: !override
      - "127.0.0.1:${APP_PORT}:3000"
```

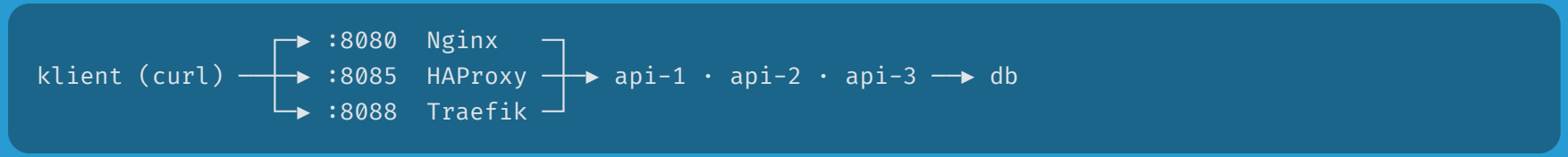
- **obraz z rejestru:** `pull` **+** `up -d --wait`
- **przypięte wersje,** `restart`**, healthchecki**
- **limity i rotacja logów**
- **sekrety jako pliki,** `.env` **poza repo**
- **publikuj tylko proxy (80/443)**
- **backup woluminów**
- **granica: jeden host → Kubernetes**

Ćwiczenie dodatkowe – pełna wersja notes-api ★

```
cd app-node
docker compose up -d --build --wait
curl -X POST localhost:8000/notes -H 'Content-Type: application/json' -d '{"text":"Pierwsza"}'
curl localhost:8000/notes
docker compose down && docker compose up -d --wait && curl localhost:8000/notes
docker compose stop db && curl localhost:8000/readyz      # 503
docker compose start db
docker compose --profile narzedzia up -d                  # http://localhost:8081
```

```
docker/13-compose-zaawansowane/compose-aplikacja.sh
```

Repliki za proxy ★




```yaml
# compose.yaml
include:
  - path:
      - ../../../app-node/compose.yaml
      - api-repliki.yaml
# api-repliki.yaml
services:
  api:
    ports: !reset []  # bez portu hosta
    deploy: { replicas: 3 }
```

```
docker compose up -d --wait
docker compose up -d --scale api=5 --wait
docker compose exec nginx nslookup api 127.0.0.11
for i in $(seq 9); do curl -s localhost:8080/ \
  | grep -o '"hostname":"[^"]*"'; done | uniq -c
```

Nginx · HAProxy · Traefik ★

```
resolver 127.0.0.11 valid=5s;
upstream api {
  zone api 64k;
  server api:3000 resolve
    max_fails=1;
}
location / {
  proxy_pass http://api;
  proxy_next_upstream
    error timeout http_503;
  proxy_read_timeout 3s;
}
```

```
resolvers docker
  nameserver dns 127.0.0.11:53
backend api
  balance roundrobin
  option httpchk GET /readyz
  option redispatch
  server-template api 5
    api:3000 check inter 2s
    resolvers docker
    init-addr none
```

```
traefik:
  command:
    - --providers.docker=true
  volumes:
    - /var/run/docker.sock:…:ro
api:
  labels:
    - traefik.enable=true
    - traefik.http.services.api
        .loadbalancer.server
        .port=3000
    - …healthcheck.path=/readyz
```

plik + DNS (pasywnie) · plik + DNS (aktywny check, statystyki :8404) · etykiety + API Dockera (dashboard :8089)

Porównanie – zmierzone ★

	Nginx (open source)	HAProxy	Traefik
odkrywanie replik	DNS (`resolve`)	DNS (`server-template`)	API Dockera
health check	pasywny	aktywny `/readyz`	aktywny `/readyz`
`--scale 3 → 5`	równo, bez restartu	równo	równo
`docker stop` repliki	ponowienie, sporadycznie 502	0 błędów	0 błędów
`docker pause` repliki	1 żądanie czeka 3 s	0 błędów	0 błędów
zastosowanie	WWW, statyczne pliki, reverse proxy	wydajny LB L4/L7	proxy kontenerów, TLS

Kubernetes: Service (L4) · Ingress / Gateway API (L7: ingress-nginx, Traefik, HAProxy) · readinessProbe

Load balancing – animacja ★



animacja: assets/animations/load-balancing.mp4

Ćwiczenie dodatkowe – load balancing ★

1. `cd docker/13-compose-zaawansowane/load-balancing && docker compose up -d --wait`
2. **rozkład ruchu na portach 8080 · 8085 · 8088 (`curl` w pętli + `uniq -c`)**
3. `docker compose up -d --scale api=5 --wait` → **statystyki HAProxy** `localhost:8404`, **dashboard Traefika** `localhost:8089`
4. `docker stop lb-api-2`, `docker pause lb-api-3` – **porównaj** `curl -w '%{time_total}'`
5. **dla chętnych:** `balance source` **w HAProxy – ten sam klient → ta sama replika**
6. `docker compose down -v`

```
docker/13-compose-zaawansowane/load-balancing/load-balancing.sh · ok. 20 min
```

14. Prywatny rejestr obrazów

registry:2 · Docker Hub · tagi · CI · podpisy · chmury

Rejestr lokalny – `registry:2`

```
docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2
docker tag moj-apache:1.0 localhost:5000/moj-apache:1.0
docker push localhost:5000/moj-apache:1.0
curl localhost:5000/v2/_catalog
curl localhost:5000/v2/moj-apache/tags/list
docker rmi localhost:5000/moj-apache:1.0 && docker pull localhost:5000/moj-apache:1.0

echo "$GITHUB_TOKEN" | docker login ghcr.io -u <login> --password-stdin
docker push ghcr.io/<login>/moj-apache:1.0
```

- **Docker Hub: limity pobrań (niższe dla anonimowych)**
- **firmowo: Harbor, ECR, Artifact Registry, ACR, GHCR, GitLab, Nexus**

Docker Hub i wersjonowanie tagów ★

Tag	Zmienny?	Do czego
`1.2.0`	nie	wdrożenie, rollback
`1.2`, `1`	tak	najnowsza poprawka
`sha-ca11883`	nie	commit źródłowy
`latest`	tak	**nie** na produkcji
`@sha256:…`	nigdy	identyczny obraz

- **Official Images, Verified Publisher, Sponsored OSS – sprawdzone źródła**
- **limity pobrań →** `docker login`**, mirror w CI**
- **obraz bazowy łata się pod tym samym tagiem →** `docker build --pull` **regularnie**
- **Dependabot / Renovate: PR z nowym** `FROM`

Metadane – etykiety OCI ★

```
LABEL org.opencontainers.image.title="notes-api" \
      org.opencontainers.image.source="https://github.com/<org>/notes-api" \
      org.opencontainers.image.licenses="MIT"
```

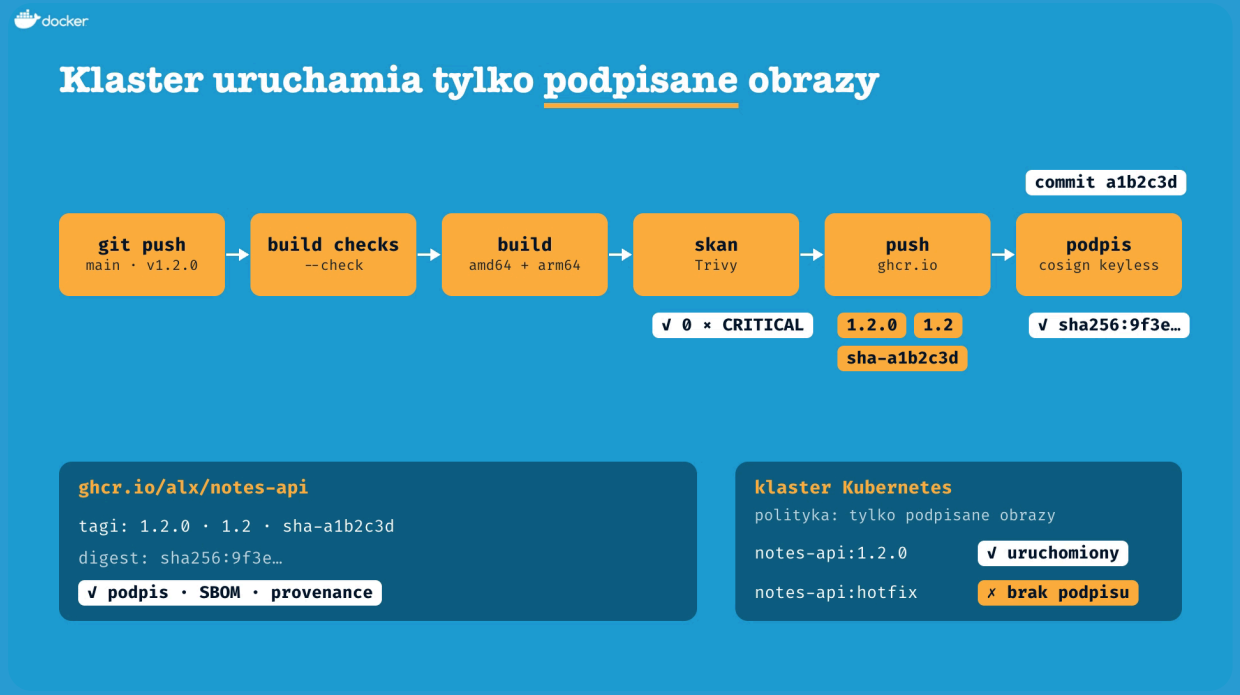
```
docker build --label org.opencontainers.image.version=1.2.0 \
  --label org.opencontainers.image.revision="$(git rev-parse --short HEAD)" -t notes-api:1.2.0 .
docker inspect --format '{{json .Config.Labels}}' notes-api:1.2.0
```

- **standardowe klucze czytają rejestry, Scout, Renovate**
- **GHCR:** `source` **= adres repo GitHub → pakiet połączony z repozytorium (uprawnienia, README)**
- `MAINTAINER` **przestarzałe →** `LABEL` **; w CI etykiety daje** `docker/metadata-action`
- **kto zbudował obraz → podpis cosign (za chwilę)**

CI – GitHub Actions ★

```yaml
on: { push: { branches: [main], tags: ["v*.*.*"] }, pull_request: }
permissions: { contents: read, packages: write, id-token: write }
steps:
  - uses: docker/setup-buildx-action@v4
  - uses: docker/build-push-action@v7          # 1. build checks
    with: { call: check }
  - uses: docker/login-action@v4               # 2. GHCR, token GITHUB_TOKEN
  - uses: docker/metadata-action@v6            # 3. tagi: 1.2.0, 1.2, main, sha-…
  # 4. skan Trivy – bramka CRITICAL/HIGH
  - uses: docker/build-push-action@v7          # 5. amd64+arm64, push, SBOM, provenance
    with:
      platforms: linux/amd64,linux/arm64
      push: true
      cache-from: type=gha
      cache-to: type=gha,mode=max
  - run: cosign sign --yes ghcr.io/…@${{ steps.build.outputs.digest }}    # 6. podpis keyless
```

CI – animacja ★



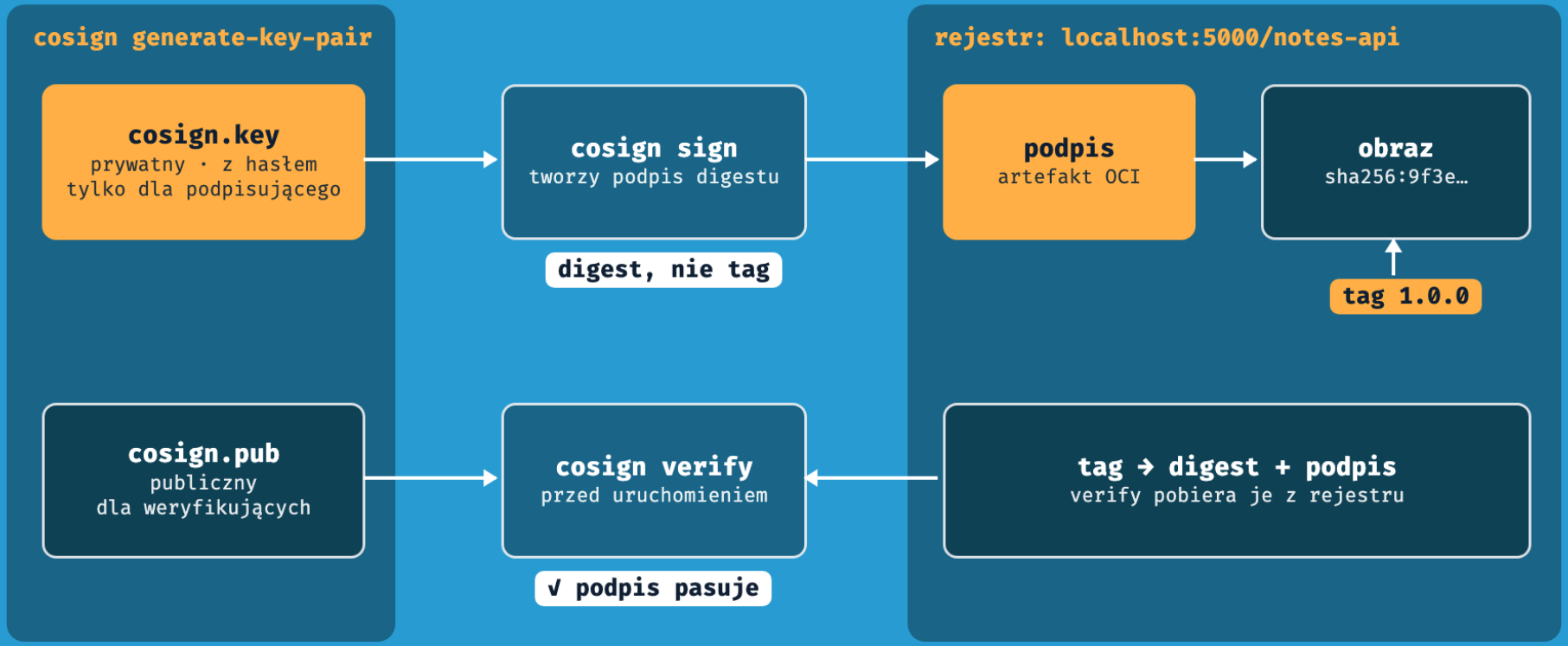
animacja: assets/animations/ci-pipeline.mp4

Podpisy obrazów – cosign ★

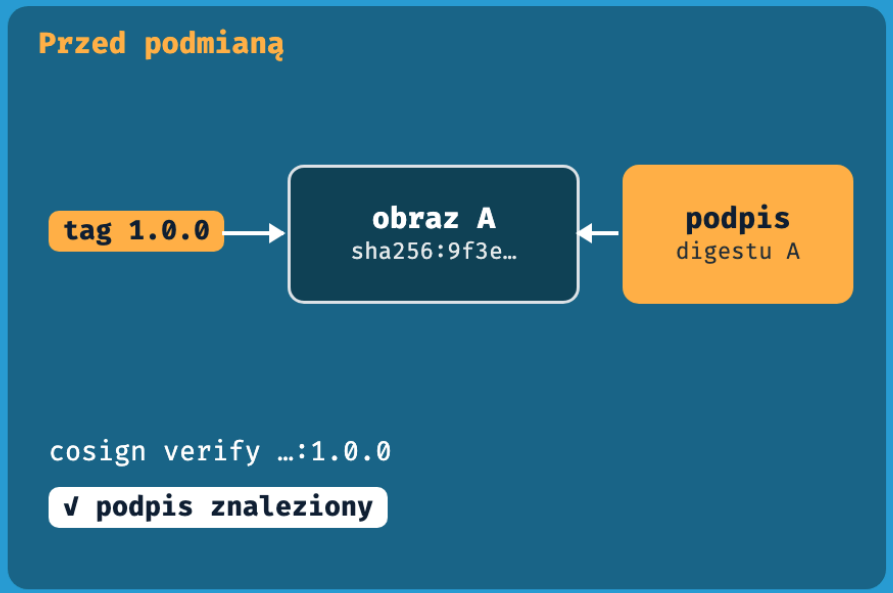
```
IMG=localhost:5000/notes-api
cosign generate-key-pair                                  # cosign.key + cosign.pub
DIGEST=$(docker buildx imagetools inspect $IMG:1.0.0 --format '{{.Manifest.Digest}}')
cosign sign --key cosign.key $IMG@$DIGEST                 # podpisujemy digest, nie tag
cosign verify --key cosign.pub $IMG:1.0.0                 # ✓
docker push $IMG:1.0.0                                    # … ktoś podmienia obraz pod tagiem
cosign verify --key cosign.pub $IMG:1.0.0                 # ✗ no signatures found
```

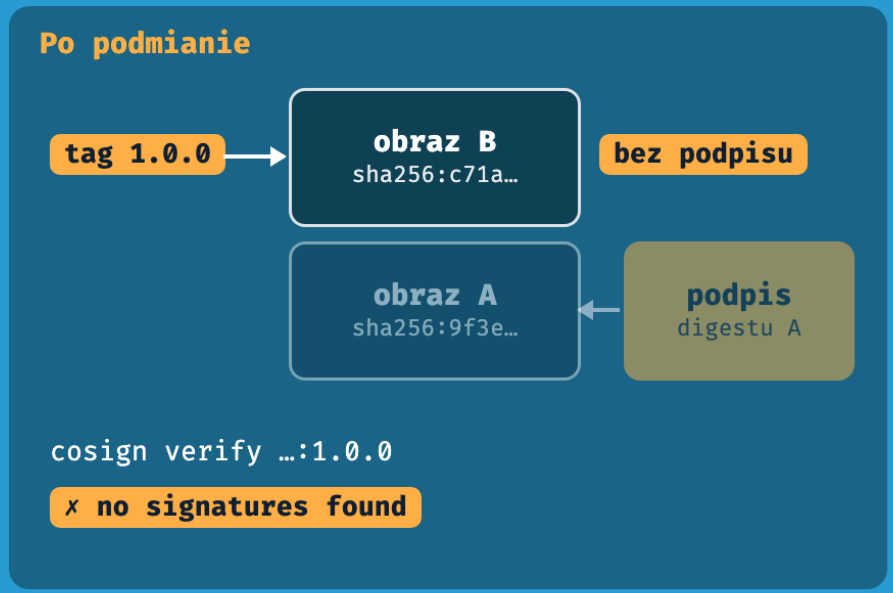
- **keyless w CI: tożsamość z OIDC, certyfikat Fulcio, log Rekor**
- **skan → SBOM → provenance → podpis; w klastrze wymusza Kyverno**
- **Docker Content Trust (Notary v1) – historyczne, wycofywane**

Prywatny podpisuje, publiczny sprawdza



Tag się przesuwa, podpis zostaje przy digeście






```
$ docker tag busybox localhost:5000/notes-api:1.0.0 && docker push localhost:5000/notes-api:1.0.0
$ cosign verify --key cosign.pub localhost:5000/notes-api:1.0.0
Error: no signatures found
```

Keyless: tożsamość zamiast klucza do pilnowania



Co sprawdza cosign verify

✓ podpis pasuje do digestu

✓ certyfikat wydało Fulcio

✓ tożsamość jest tą, której ufamy

✓ podpis jest wpisany do Rekora

```
$ cosign sign --yes ghcr.io/alx/notes-api@sha256:9f3e…

$ cosign verify ghcr.io/alx/notes-api:1.2.0 \
    --certificate-identity-regexp \
      'https://github.com/alx/notes-api/.*' \
    --certificate-oidc-issuer \
      https://token.actions.githubusercontent.com
```

Docker a chmury ★

Model	Przykłady	Dostarczasz
rejestr	ECR · Artifact Registry · ACR	obraz
kontenery bez serwerów	ECS Fargate · Cloud Run · Azure Container Apps	obraz + port, zmienne, CPU/RAM
zarządzany Kubernetes	EKS · GKE · AKS	obraz + manifesty
VM z Dockerem	EC2 · Compute Engine · Azure VM	obraz + `compose.yaml`

- **obrazy** `amd64` **+** `arm64` **(Graviton, Axion) · aplikacja bezstanowa · logi na stdout**
- **logowanie do rejestru chmury:** `aws ecr get-login-password`, `gcloud auth configure-docker`, `az acr login`

15. Troubleshooting

logi • kody wyjścia • sieć • miejsce na dysku

Najczęstsze problemy

```
docker logs <k>
docker inspect -f '{{.State.ExitCode}}' <k>
docker run -it --rm --entrypoint sh <obraz>
docker network inspect <sieć>
docker debug <k>        # obraz bez powłoki
docker system df
docker builder prune
```

Kod	Znaczenie
0	OK
1	błąd aplikacji
125	błąd `docker run`
126	nie można uruchomić
127	brak polecenia
137	SIGKILL (OOM, timeout stop)
143	SIGTERM

16. AI w pracy z Dockerem

dodatek · generuj szybciej · weryfikuj zawsze

Typowy wynik krótkiego promptu ★

```
FROM node:latest
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
ENV PGPASSWORD=haslo-w-obrazie
EXPOSE 8080
CMD npm start
```

- `latest`, **jeden etap → 1,3 GB**
- `COPY . .` **przed instalacją**
- `npm install` **zamiast** `npm ci`
- **hasło w** `ENV` **→ widoczne w** `history`
- **root, zły port, forma shell**

Weryfikacja i dobre praktyki ★

```
docker init                                                    # szablon bez AI
docker build --check -f wygenerowany/Dockerfile ../../app-node      # build checks
docker run --rm -i hadolint/hadolint < wygenerowany/Dockerfile # linter
docker scout quickview <obraz>                                 # podatności
docker compose config                                          # walidacja compose.yaml
```

- **podawaj kontekst: pliki, wersje, wymagania**
- **proś o uzasadnienie i wskazanie niepewności**
- **uwaga na halucynacje: nieistniejące tagi, flagi, przestarzałe instrukcje**
- **nie wklejaj sekretów ani kodu objętego tajemnicą**
- **buduj, uruchamiaj, testuj – jak kod pisany ręcznie**

Docker Skills – wiedza dla agenta ★

Oficjalne instrukcje Dockera dla agentów AI: agent wczytuje `SKILL.md`, zamiast pisać z pamięci.

```
/plugin marketplace add docker/skills     # Claude Code, w sesji agenta
/plugin install docker-skills@docker
npx skills add docker/skills              # inne agenty – skills CLI
```

- `docker-build-strategies` • `docker-compose-patterns` • `docker-project-foundations`
- `docker-destructive-guardrails` **– hamulec przed poleceniami usuwającymi dane**
- `docker-docs` **– aktualna dokumentacja zamiast pamięci modelu**
- **skill to kod zewnętrzny: przejrzyj, przypnij tag; weryfikacja zostaje**

Od Dockera do Kubernetesa

Docker	Kubernetes
obraz, rejestr	te same obrazy OCI
kontener	kontener w Podzie
`--restart`, wiele replik	Deployment / ReplicaSet
`HEALTHCHECK`, `/healthz`, `/readyz`	`livenessProbe`, `readinessProbe`
`-p`, DNS w sieci	Service, Ingress, DNS klastra
`--scale` + Nginx / HAProxy / Traefik	`replicas` + Service + Ingress
wolumin	PersistentVolume / PVC
`.env` · `secrets:`	ConfigMap · Secret
`--memory`, `--cpus`	`resources.requests` / `limits`
`compose.yaml`	manifesty YAML, Helm

Pytania?

obiad do 13:45 · dalej: Kubernetes